

import necessary libraries

```
In [1]: !pip install seaborn
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
!pip install scipy
from scipy.stats import ttest_ind
```

WARNING: You are using pip version 21.1.3; however, version 25.3 is available.
You should consider upgrading via the 'c:\users\vsvid\appdata\local\programs\python\python39\python.exe -m pip install --upgrade pip' command.

Requirement already satisfied: seaborn in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (0.13.2)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from seaborn) (3.9.4)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from seaborn) (2.0.2)

Requirement already satisfied: pandas>=1.2 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from seaborn) (2.3.3)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.7)

Requirement already satisfied: importlib-resources>=3.2.0 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (6.5.2)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: pillow>=8 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)

Requirement already satisfied: packaging>=20.0 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.60.2)

Requirement already satisfied: cycler>=0.10 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.0)

Requirement already satisfied: zipp>=3.1.0 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from importlib-resources>=3.2.0->matplotlib!=3.6.1,>=3.4->seaborn) (3.23.0)

Requirement already satisfied: pytz>=2020.1 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from pandas>=1.2->seaborn) (2025.3)

Requirement already satisfied: six>=1.5 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

Requirement already satisfied: scipy in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (1.13.1)

WARNING: You are using pip version 21.1.3; however, version 25.3 is available.
You should consider upgrading via the 'c:\users\vsvid\appdata\local\programs\python\python39\python.exe -m pip install --upgrade pip' command.

Requirement already satisfied: numpy<2.3,>=1.22.4 in c:\users\vsvid\appdata\local\programs\python\python39\lib\site-packages (from scipy) (2.0.2)

Get dataset 1 from folder

```
In [2]: df_customer=pd.read_csv("D:\Vidhya Learning\Quantium_Forage Virtual Job Sim\QVI_purchase_behaviour.csv")
df_customer.head() ### display 5rows
```

Out[2]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

```
In [3]: df_customer.info() ## display details

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR   72637 non-null  int64
1   LIFESTAGE        72637 non-null  object
2   PREMIUM_CUSTOMER 72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
```

```
In [4]: df_customer.describe()
```

```
Out[4]:
```

	LYLTY_CARD_NBR
count	7.263700e+04
mean	1.361859e+05
std	8.989293e+04
min	1.000000e+03
25%	6.620200e+04
50%	1.340400e+05
75%	2.033750e+05
max	2.373711e+06

Get dataset 2 from folder

```
In [5]: df_transaction=pd.read_excel("D:\Vidhya Learning\Quantium Forage Virtual Job Sim\QVI_transaction_data.xlsx")
df_transaction.head()    ### display 5 rows
```

```
Out[5]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8

```
In [6]: df_transaction.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   DATE             264836 non-null int64
1   STORE_NBR        264836 non-null int64
2   LYLTY_CARD_NBR   264836 non-null int64
3   TXN_ID           264836 non-null int64
4   PROD_NBR         264836 non-null int64
5   PROD_NAME        264836 non-null object
6   PROD_QTY         264836 non-null int64
7   TOT_SALES        264836 non-null float64
dtypes: float64(1), int64(6), object(1)
memory usage: 16.2+ MB
```

```
In [7]: df_transaction.describe()
```

```
Out[7]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES
count	264836.000000	264836.000000	2.648360e+05	2.648360e+05	264836.000000	264836.000000	264836.000000
mean	43464.036260	135.08011	1.355495e+05	1.351583e+05	56.583157	1.907309	7.304200
std	105.389282	76.78418	8.057998e+04	7.813303e+04	32.826638	0.643654	3.083226
min	43282.000000	1.00000	1.000000e+03	1.000000e+00	1.000000	1.000000	1.500000
25%	43373.000000	70.00000	7.002100e+04	6.760150e+04	28.000000	2.000000	5.400000
50%	43464.000000	130.00000	1.303575e+05	1.351375e+05	56.000000	2.000000	7.400000
75%	43555.000000	203.00000	2.030942e+05	2.027012e+05	85.000000	2.000000	9.200000
max	43646.000000	272.00000	2.373711e+06	2.415841e+06	114.000000	200.000000	650.000000

Merge two datasets based on Loyalty card number which is common

```
In [8]: df= pd.merge(df_transaction, df_customer, on="LYLTY_CARD_NBR")

df.head()
```

Out[8]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	LIFESTAGE	PI
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	

Change column headers

In [9]:

```
df = df.rename(columns={"LYLTY_CARD_NBR" : "Card No.",
                        "LIFESTAGE" : "Group",
                        "DATE" : "Date",
                        "PREMIUM_CUSTOMER" : "Subscription",
                        "STORE_NBR" : "Store Id",
                        "TXN_ID" : "Transaction Id",
                        "PROD_NBR" : "Product Id",
                        "PROD_NAME" : "Product",
                        "PROD_QTY" : "Quantity",
                        "TOT_SALES" : "Sales",
                        })

df.head()
```

Out[9]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget

Check data types

In [10]:

```
df.dtypes
```

Out[10]:

```
Date                int64
Store Id            int64
Card No.            int64
Transaction Id      int64
Product Id          int64
Product             object
Quantity            int64
Sales               float64
Group               object
Subscription         object
dtype: object
```

Change date to date format based on exceldate structure whoch starts from 30-12-1899

In [11]:

```
df["Date"] = pd.to_datetime(df["Date"], unit="D", origin="1899-12-30")
df_transaction.head()
```

Out[11]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8

Check for null value

```
In [12]: df.isnull().sum()
```

```
Out[12]: Date                0
Store Id                    0
Card No.                    0
Transaction Id              0
Product Id                  0
Product                     0
Quantity                    0
Sales                       0
Group                       0
Subscription                0
dtype: int64
```

Check for unique value that is what products are sold

```
In [13]: df["Product"].unique()
```

```
Out[13]: array(['Natural Chip          Compny SeaSalt175g',
                'CCs Nacho Cheese      175g',
                'Smiths Crinkle Cut   Chips Chicken 170g',
                'Smiths Chip Thinly  S/Cream&Onion 175g',
                'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
                'Old El Paso Salsa   Dip Tomato Mild 300g',
                'Smiths Crinkle Chips Salt & Vinegar 330g',
                'Grain Waves          Sweet Chilli 210g',
                'Doritos Corn Chip Mexican Jalapeno 150g',
                'Grain Waves Sour   Cream&Chives 210G',
                'Kettle Sensations  Siracha Lime 150g',
                'Twisties Cheese    270g', 'WW Crinkle Cut      Chicken 175g',
                'Thins Chips Light&  Tangy 175g', 'CCs Original 175g',
                'Burger Rings 220g', 'NCC Sour Cream &   Garden Chives 175g',
                'Doritos Corn Chip Southern Chicken 150g',
                'Cheezels Cheese Box 125g', 'Smiths Crinkle      Original 330g',
                'Infzns Crn Crnchers Tangy Gcamole 110g',
                'Kettle Sea Salt    And Vinegar 175g',
                'Smiths Chip Thinly  Cut Original 175g', 'Kettle Original 175g',
                'Red Rock Deli Thai Chilli&Lime 150g',
                'Pringles Sthrn FriedChicken 134g', 'Pringles Sweet&Spcy BBQ 134g',
                'Red Rock Deli SR   Salsa & Mzzrlla 150g',
                'Thins Chips          Originl saltd 175g',
                'Red Rock Deli Sp    Salt & Truffle 150G',
                'Smiths Thinly      Swt Chli&S/Cream175G', 'Kettle Chilli 175g',
                'Doritos Mexicana    170g',
                'Smiths Crinkle Cut   French OnionDip 150g',
                'Natural ChipCo       Hony Soy Chckn175g',
                'Dorito Corn Chp      Supreme 380g', 'Twisties Chicken270g',
                'Smiths Thinly Cut    Roast Chicken 175g',
                'Smiths Crinkle Cut   Tomato Salsa 150g',
                'Kettle Mozzarella    Basil & Pesto 175g',
                'Infuzions Thai SweetChili PotatoMix 110g',
                'Kettle Sensations    Camembert & Fig 150g',
                'Smith Crinkle Cut    Mac N Cheese 150g',
                'Kettle Honey Soy     Chicken 175g',
                'Thins Chips Seasonedchicken 175g',
                'Smiths Crinkle Cut   Salt & Vinegar 170g',
                'Infuzions BBQ Rib    Prawn Crackers 110g',
                'GrnWves Plus Btroot & Chilli Jam 180g',
                'Tyrrells Crisps      Lightly Salted 165g',
                'Kettle Sweet Chilli   And Sour Cream 175g',
                'Doritos Salsa        Medium 300g', 'Kettle 135g Swt Pot Sea Salt',
                'Pringles SourCream   Onion 134g',
                'Doritos Corn Chips    Original 170g',
                'Twisties Cheese      Burger 250g',
                'Old El Paso Salsa     Dip Chnky Tom Ht300g',
                'Cobs Popd Swt/Chlli   &Sr/Cream Chips 110g',
                'Woolworths Mild      Salsa 300g',
                'Natural Chip Co       Tmato Hrb&Spce 175g',
                'Smiths Crinkle Cut    Chips Original 170g',
                'Cobs Popd Sea Salt     Chips 110g',
                'Smiths Crinkle Cut    Chips Chs&Onion170g',
                'French Fries Potato   Chips 175g',
                'Old El Paso Salsa     Dip Tomato Med 300g',
                'Doritos Corn Chips    Cheese Supreme 170g',
                'Pringles Original     Crisps 134g',
                'RRD Chilli&          Coconut 150g',
                'WW Original Corn      Chips 200g',
                'Thins Potato Chips    Hot & Spicy 175g',
                'Cobs Popd Sour Crm     &Chives Chips 110g',
                'Smiths Crnkle Chip     Orgnl Big Bag 380g',
                'Doritos Corn Chips    Nacho Cheese 170g',
                'Kettle Sensations     BBQ&Maple 150g',
                'WW D/Style Chip       Sea Salt 200g',
```

```
'Pringles Chicken    Salt Crips 134g',
'WW Original Stacked Chips 160g',
'Smiths Chip Thinly  CutSalt/Vinegr175g', 'Cheezels Cheese 330g',
'Tostitos Lightly    Salted 175g',
'Thins Chips Salt &  Vinegar 175g',
'Smiths Crinkle Cut  Chips Barbecue 170g', 'Cheetos Puffs 165g',
'RRD Sweet Chilli &  Sour Cream 165g',
'WW Crinkle Cut      Original 175g',
'Tostitos Splash Of  Lime 175g', 'Woolworths Medium Salsa 300g',
'Kettle Tortilla ChpsBtroot&Ricotta 150g',
'CCs Tasty Cheese    175g', 'Woolworths Cheese Rings 190g',
'Tostitos Smoked     Chipotle 175g', 'Pringles Barbeque 134g',
'WW Supreme Cheese   Corn Chips 200g',
'Pringles Mystery    Flavour 134g',
'Tyrrells Crisps     Ched & Chives 165g',
'Snbts Whlgrn Crisps Cheddr&Mstrd 90g',
'Cheetos Chs & Bacon Balls 190g', 'Pringles Slt Vingar 134g',
'Infuzions SourCream&Herbs Veg Strws 110g',
'Kettle Tortilla ChpsFeta&Garlic 150g',
'Infuzions Mango     Chutny Papadums 70g',
'RRD Steak &         Chimuchurri 150g',
'RRD Honey Soy       Chicken 165g',
'Sunbites Whlegrn    Crisps Frch/Onin 90g',
'RRD Salt & Vinegar   165g', 'Doritos Cheese Supreme 330g',
'Smiths Crinkle Cut  Snag&Sauce 150g',
'WW Sour Cream &OnionStacked Chips 160g',
'RRD Lime & Pepper    165g',
'Natural ChipCo Sea  Salt & Vinegr 175g',
'Red Rock Deli Chikn&Garlic Aioli 150g',
'RRD SR Slow Rst     Pork Belly 150g', 'RRD Pc Sea Salt 165g',
'Smith Crinkle Cut    Bolognese 150g', 'Doritos Salsa Mild 300g'],
dtype=object)
```

```
In [14]: split_prods = df["Product"].str.replace(r"([0-9]+[gG])", "").str.replace(r"^\w", " ").str.split()
split_prods
```

```
Out[14]: 0          [Natural, Chip, Compny, SeaSalt175g]
1          [CCs, Nacho, Cheese, 175g]
2          [Smiths, Crinkle, Cut, Chips, Chicken, 170g]
3          [Smiths, Chip, Thinly, S/Cream&Onion, 175g]
4          [Kettle, Tortilla, ChpsHny&Jlpno, Chili, 150g]
...
264831     [Kettle, Sweet, Chilli, And, Sour, Cream, 175g]
264832     [Tostitos, Splash, Of, Lime, 175g]
264833     [Doritos, Mexicana, 170g]
264834     [Doritos, Corn, Chip, Mexican, Jalapeno, 150g]
264835     [Tostitos, Splash, Of, Lime, 175g]
Name: Product, Length: 264836, dtype: object
```

Count no of occurance of each word

```
In [15]: word_counts = {}
def count_words(line) :
    for word in line :
        if word not in word_counts :
            word_counts[word] = 1
        else :
            word_counts[word] += 1
split_prods.apply(lambda line : count_words(line))
print(pd.Series(word_counts).sort_values(ascending=False))
```

```
175g      60561
Chips      49770
150g      41633
Kettle     41288
&          35565
...
Whlegrn    1432
Pc          1431
NCC         1419
Garden      1419
Fries       1418
Length: 220, dtype: int64
```

remove special characters

```
In [16]: df['Product'] = df['Product'].astype(str).str.replace(r"([0-9]+[gG])", "").str.replace(r"^\w", " ", regex=True)
df['Product']
```

```
Out[16]: 0      Natural Chip      Compny SeaSalt175g
1              CCs Nacho Cheese      175g
2      Smiths Crinkle Cut  Chips Chicken 170g
3      Smiths Chip Thinly  S Cream Onion 175g
4      Kettle Tortilla ChpsHny Jlpno Chili 150g
...
264831      Kettle Sweet Chilli And Sour Cream 175g
264832              Tostitos Splash Of  Lime 175g
264833              Doritos Mexicana      170g
264834      Doritos Corn Chip Mexican Jalapeno 150g
264835              Tostitos Splash Of  Lime 175g
Name: Product, Length: 264836, dtype: object
```

Product summary on frequency of each product

```
In [17]: freq =df['Product'].value_counts().reset_index()
freq.columns = ['Product', 'Frequency']
freq
```

Out[17]:

	Product	Frequency
0	Kettle Mozzarella Basil Pesto 175g	3304
1	Kettle Tortilla ChpsHny Jlpno Chili 150g	3296
2	Cobs Popd Swt Chlli Sr Cream Chips 110g	3269
3	Tyrrells Crisps Ched Chives 165g	3268
4	Cobs Popd Sea Salt Chips 110g	3265
...	...	...
109	RRD Pc Sea Salt 165g	1431
110	Woolworths Medium Salsa 300g	1430
111	NCC Sour Cream Garden Chives 175g	1419
112	French Fries Potato Chips 175g	1418
113	WW Crinkle Cut Original 175g	1410

114 rows × 2 columns

Find number of products sold

```
In [18]: uni_prod_name=df['Product'].unique()
uni_prod_name
```

```
Out[18]: array(['Natural Chip      Compny SeaSalt175g',
      'CCs Nacho Cheese      175g',
      'Smiths Crinkle Cut  Chips Chicken 170g',
      'Smiths Chip Thinly  S Cream Onion 175g',
      'Kettle Tortilla ChpsHny Jlpno Chili 150g',
      'Old El Paso Salsa  Dip Tomato Mild 300g',
      'Smiths Crinkle Chips Salt  Vinegar 330g',
      'Grain Waves      Sweet Chilli 210g',
      'Doritos Corn Chip Mexican Jalapeno 150g',
      'Grain Waves Sour  Cream Chives 210g',
      'Kettle Sensations  Siracha Lime 150g',
      'Twisties Cheese      270g', 'WW Crinkle Cut      Chicken 175g',
      'Thins Chips Light  Tangy 175g', 'CCs Original 175g',
      'Burger Rings 220g', 'NCC Sour Cream      Garden Chives 175g',
      'Doritos Corn Chip Southern Chicken 150g',
      'Cheezels Cheese Box 125g', 'Smiths Crinkle      Original 330g',
      'Infzns Crn Crnchers Tangy Gcamole 110g',
      'Kettle Sea Salt      And Vinegar 175g',
      'Smiths Chip Thinly  Cut Original 175g', 'Kettle Original 175g',
      'Red Rock Deli Thai  Chilli Lime 150g',
      'Pringles Sthrn FriedChicken 134g', 'Pringles Sweet Spcy BBQ 134g',
      'Red Rock Deli SR  Salsa  Mzzrlla 150g',
      'Thins Chips      Originl salted 175g',
      'Red Rock Deli Sp  Salt  Truffle 150g',
      'Smiths Thinly  Swt Chli S Cream175g', 'Kettle Chilli 175g',
      'Doritos Mexicana      170g',
      'Smiths Crinkle Cut  French OnionDip 150g',
      'Natural ChipCo      Hony Soy Chckn175g',
      'Dorito Corn Chp      Supreme 380g', 'Twisties Chicken270g',
      'Smiths Thinly Cut  Roast Chicken 175g',
      'Smiths Crinkle Cut  Tomato Salsa 150g',
      'Kettle Mozzarella  Basil  Pesto 175g',
      'Infuzions Thai SweetChili PotatoMix 110g',
      'Kettle Sensations  Camembert  Fig 150g',
      'Smith Crinkle Cut  Mac N Cheese 150g',
      'Kettle Honey Soy      Chicken 175g',
      'Thins Chips Seasonedchicken 175g',
      'Smiths Crinkle Cut  Salt  Vinegar 170g',
```

```

'Infuzions BBQ Rib Prawn Crackers 110g',
'GrnWves Plus Btroot Chilli Jam 180g',
'Tyrrells Crisps Lightly Salted 165g',
'Kettle Sweet Chilli And Sour Cream 175g',
'Doritos Salsa Medium 300g', 'Kettle 135g Swt Pot Sea Salt',
'Pringles SourCream Onion 134g',
'Doritos Corn Chips Original 170g',
'Twisties Cheese Burger 250g',
'Old El Paso Salsa Dip Chnky Tom Ht300g',
'Cobs Popd Swt Chlli Sr Cream Chips 110g',
'Woolworths Mild Salsa 300g',
'Natural Chip Co Tmato Hrb Spce 175g',
'Smiths Crinkle Cut Chips Original 170g',
'Cobs Popd Sea Salt Chips 110g',
'Smiths Crinkle Cut Chips Chs Onion170g',
'French Fries Potato Chips 175g',
'Old El Paso Salsa Dip Tomato Med 300g',
'Doritos Corn Chips Cheese Supreme 170g',
'Pringles Original Crisps 134g',
'RRD Chilli Coconut 150g',
'WW Original Corn Chips 200g',
'Thins Potato Chips Hot Spicy 175g',
'Cobs Popd Sour Crm Chives Chips 110g',
'Smiths Crnkle Chip Orgnl Big Bag 380g',
'Doritos Corn Chips Nacho Cheese 170g',
'Kettle Sensations BBQ Maple 150g',
'WW D Style Chip Sea Salt 200g',
'Pringles Chicken Salt Crips 134g',
'WW Original Stacked Chips 160g',
'Smiths Chip Thinly CutSalt Vinegr175g', 'Cheezels Cheese 330g',
'Tostitos Lightly Salted 175g',
'Thins Chips Salt Vinegar 175g',
'Smiths Crinkle Cut Chips Barbecue 170g', 'Cheetos Puffs 165g',
'RRD Sweet Chilli Sour Cream 165g',
'WW Crinkle Cut Original 175g',
'Tostitos Splash Of Lime 175g', 'Woolworths Medium Salsa 300g',
'Kettle Tortilla ChpsBtroot Ricotta 150g',
'CCs Tasty Cheese 175g', 'Woolworths Cheese Rings 190g',
'Tostitos Smoked Chipotle 175g', 'Pringles Barbeque 134g',
'WW Supreme Cheese Corn Chips 200g',
'Pringles Mystery Flavour 134g',
'Tyrrells Crisps Ched Chives 165g',
'Snbts Whlgrn Crisps Cheddr Mstrd 90g',
'Cheetos Chs Bacon Balls 190g', 'Pringles Slt Vingar 134g',
'Infuzions SourCream Herbs Veg Strws 110g',
'Kettle Tortilla ChpsFeta Garlic 150g',
'Infuzions Mango Chutny Papadums 70g',
'RRD Steak Chimuchurri 150g',
'RRD Honey Soy Chicken 165g',
'Sunbites Whlegrn Crisps Frch Onin 90g',
'RRD Salt Vinegar 165g', 'Doritos Cheese Supreme 330g',
'Smiths Crinkle Cut Snag Sauce 150g',
'WW Sour Cream OnionStacked Chips 160g',
'RRD Lime Pepper 165g',
'Natural ChipCo Sea Salt Vinegr 175g',
'Red Rock Deli Chikn Garlic Aioli 150g',
'RRD SR Slow Rst Pork Belly 150g', 'RRD Pc Sea Salt 165g',
'Smith Crinkle Cut Bolognese 150g', 'Doritos Salsa Mild 300g'],
dtype=object)

```

Remove salsa products as it is irrelevant

```

In [19]: df = df[~df["Product"].str.contains(r"[Ss]alsa")]
df.head()

```

```
Out[19]:
```

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S Cream Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget

```

In [20]: df.describe()

```

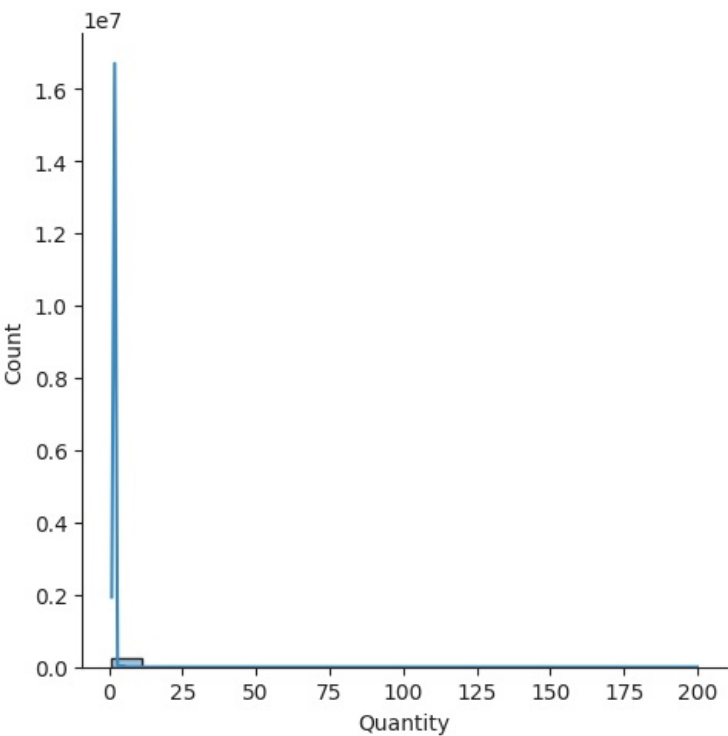
Out[20]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Quantity	Sales
count	246742	246742.000000	2.467420e+05	2.467420e+05	246742.000000	246742.000000	246742.000000
mean	2018-12-30 01:19:01.211467520	135.051098	1.355310e+05	1.351311e+05	56.351789	1.908062	7.321322
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.000000	1.700000
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	6.756925e+04	26.000000	2.000000	5.800000
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	1.351830e+05	53.000000	2.000000	7.400000
75%	2019-03-31 00:00:00	203.000000	2.030840e+05	2.026538e+05	87.000000	2.000000	8.800000
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	200.000000	650.000000
std	NaN	76.787096	8.071528e+04	7.814772e+04	33.695428	0.659831	3.077828

Find outliers in quantity

```
In [21]: sns.displot(df.Quantity, kde=True)

Out[21]: <seaborn.axisgrid.FacetGrid at 0x1f3371e1a30>
```



value 200 in quantity is found as outlier

Sort in descending order to find the customer who is in quantity outlier so as to find other transactions of that customer.

```
In [22]: df.sort_values(by = "Quantity", ascending = False).head()

Out[22]:
```

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription
69762	2018-08-19	226	226000	226201	4	Dorito Corn Chp Supreme 380g	200	650.0	OLDER FAMILIES	Premium
69763	2019-05-20	226	226000	226210	4	Dorito Corn Chp Supreme 380g	200	650.0	OLDER FAMILIES	Premium
32142	2018-08-17	39	39183	35744	81	Pringles Original Crisps 134g	5	18.5	OLDER FAMILIES	Budget
69596	2019-05-15	110	110250	113064	81	Pringles Original Crisps 134g	5	18.5	OLDER FAMILIES	Premium
72	2018-08-19	96	96203	96025	7	Smiths Crinkle Original 330g	5	28.5	MIDAGE SINGLES/COUPLES	Budget

Only Card No.226000 has purchased bulk quantity that too only twice hence can be removed if any other frequent small purchases not found

```
In [23]: len(df[df["Card No."] == 226000]) ## find no of times Card No.226000 has purchased

Out[23]: 2
```

Outliers removal

--


```
In [24]: df = df[df["Quantity"] < 6] ## Keep only quantity which is less than 6 or less than equal to 5
df.head()
```

Out[24]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S Cream Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget

another method to remove outlier df=df[df['Quantity'] < 10] ## mean of prod_qty was 1.9 taken for maximum of walues less than 10(comparing 1.9mean and max vale 200)Find transaction count by date

```
In [25]: trans_by_date = df["Date"].value_counts()
trans_by_date
```

Out[25]:

Date	
2018-12-24	865
2018-12-23	853
2018-12-22	840
2018-12-19	839
2018-12-20	808
...	
2019-06-24	612
2018-10-18	611
2018-11-25	610
2018-09-22	609
2019-06-13	607
Name: count, Length: 364, dtype: int64	

Find missing date

```
In [26]: pd.date_range(start=df["Date"].min(), end=df["Date"].max()).difference(df["Date"])
```

```
Out[26]: DatetimeIndex(['2018-12-25'], dtype='datetime64[ns]', freq='D')
```

Fill missing date to data frame

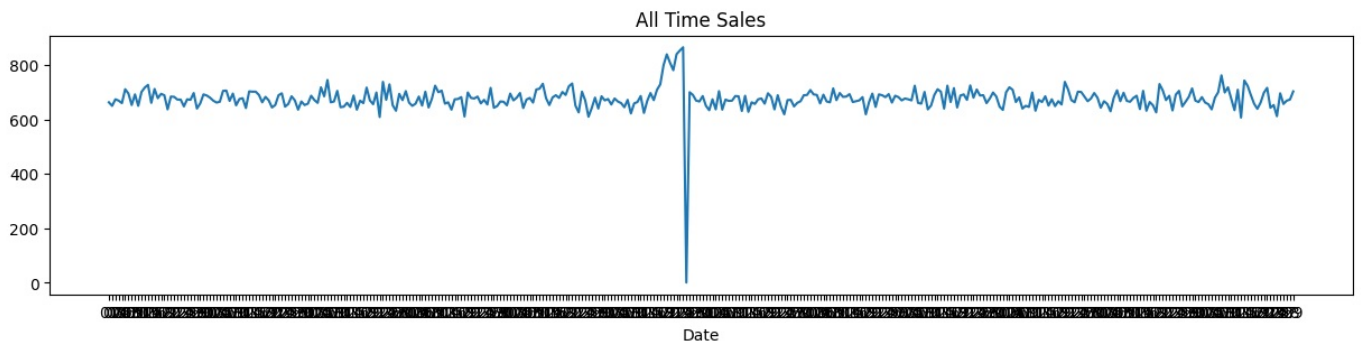
```
In [27]: merged_df = pd.merge(pd.Series(pd.date_range(start=df["Date"].min(), end=df["Date"].max()), name="Date"), df, on="Date")
merged_df.head()
```

Out[27]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription
0	2018-07-01	47.0	47142.0	42540.0	14.0	Smiths Crnkle Chip Orgnl Big Bag 380g	2.0	11.8	MIDAGE SINGLES/COUPLES	Budget
1	2018-07-01	55.0	55073.0	48884.0	99.0	Pringles Sthrn FriedChicken 134g	2.0	7.4	MIDAGE SINGLES/COUPLES	Budget
2	2018-07-01	55.0	55073.0	48884.0	91.0	CCs Tasty Cheese 175g	2.0	4.2	MIDAGE SINGLES/COUPLES	Budget
3	2018-07-01	58.0	58351.0	54374.0	102.0	Kettle Mozzarella Basil Pesto 175g	2.0	10.8	MIDAGE SINGLES/COUPLES	Budget
4	2018-07-01	68.0	68193.0	65598.0	44.0	Thins Chips Light Tangy 175g	2.0	6.6	MIDAGE SINGLES/COUPLES	Budget

Transactions over time

```
In [28]: trans_by_date = merged_df["Date"].value_counts()
all_time = trans_by_date[(trans_by_date.index >= pd.Timestamp(2018,7,1)) & (trans_by_date.index < pd.Timestamp(2018,7,1))]
all_time.index = all_time.index.strftime('%d')
ax = all_time.plot(figsize=(15,3))
ax.set_xticks(np.arange(len(all_time)))
ax.set_xticklabels(all_time.index)
plt.title("All Time Sales")
plt.savefig("All Time Sales.png", bbox_inches="tight")
plt.show()
```



Insight: Transaction is high a period and suddenly falls in another period say month

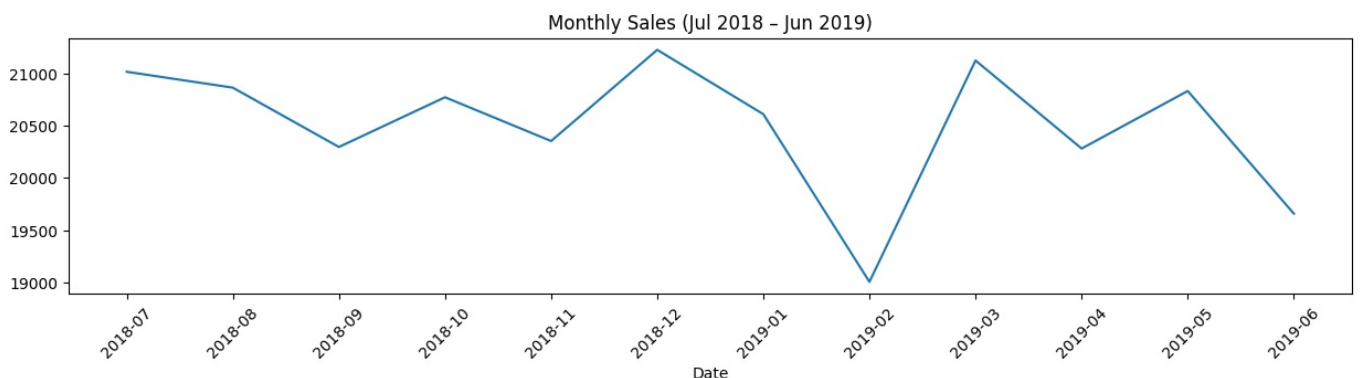
```
In [29]: # Count transactions by date
trans_by_date = merged_df["Date"].value_counts()

# Filter by date range
all_time = trans_by_date[
    (trans_by_date.index >= pd.Timestamp(2018,7,1)) &
    (trans_by_date.index < pd.Timestamp(2019,6,30))
].sort_index()

# Group by month
monthly = all_time.groupby(all_time.index.to_period("M")).sum()

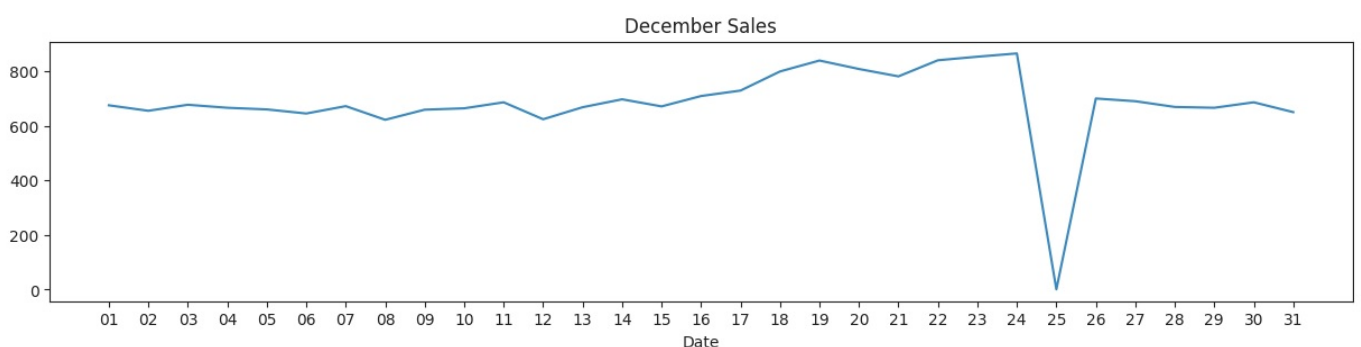
# Format index as YYYY-MM for x-axis
monthly.index = monthly.index.strftime('%Y-%m')

# Plot
ax = monthly.plot(figsize=(15,3))
ax.set_xticks(np.arange(len(monthly)))
ax.set_xticklabels(monthly.index, rotation=45)
plt.title("Monthly Sales (Jul 2018 – Jun 2019)")
plt.savefig("Monthly Sales.png", bbox_inches="tight")
plt.show()
```



Insight: Sales are in peak during december and drops in februaryDecember sales graph

```
In [30]: trans_by_date = merged_df["Date"].value_counts()
dec = trans_by_date[(trans_by_date.index >= pd.Timestamp(2018,12,1)) & (trans_by_date.index < pd.Timestamp(2019,1,1))]
dec.index = dec.index.strftime('%d')
ax = dec.plot(figsize=(15,3))
ax.set_xticks(np.arange(len(dec)))
ax.set_xticklabels(dec.index)
plt.title("December Sales")
plt.savefig("December Sales.png", bbox_inches="tight")
plt.show()
```



Insight: Sales are increasing towards Dec 24 and had a sudden fall on Dec 25 which may be because the shop is closed due to Christmas.

```
In [31]: df["Product"] = df["Product"].str.replace(r"(\d+)\s*[gG]", r"\1g", regex=True) # Ensure no spaces between the number and the unit
pack_size = df["Product"].str.extract(r"(\d+)[gG]")[0].astype(float) # Extract the numeric part before "g" or "G"
print(pack_size)
```

```
0      175.0
1      175.0
2      170.0
3      175.0
4      150.0
...
264831  175.0
264832  175.0
264833  170.0
264834  150.0
264835  175.0
Name: 0, Length: 246740, dtype: float64
```

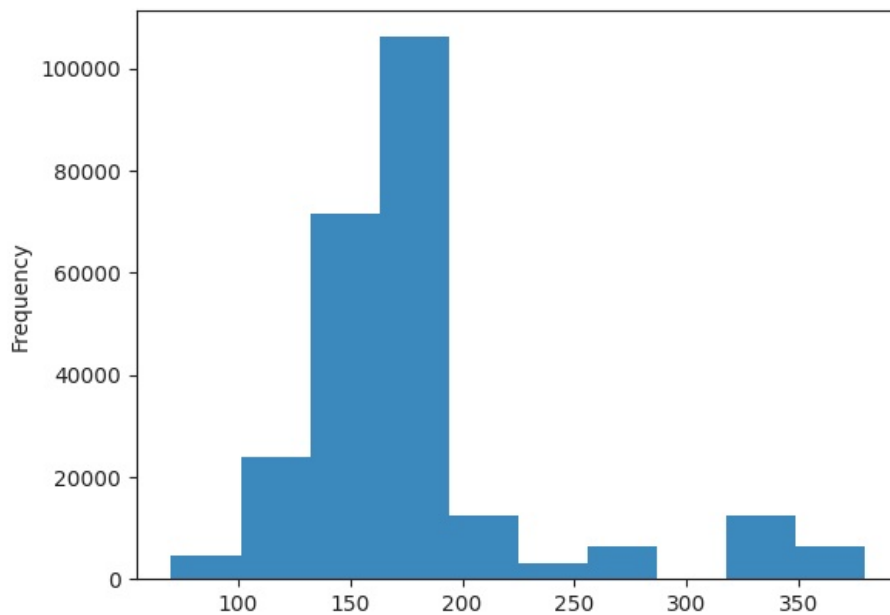
Extract pack size from Product name

```
In [32]: pack_size.describe()
```

```
Out[32]: count    246740.000000
mean      175.583521
std       59.432118
min       70.000000
25%      150.000000
50%      170.000000
75%      175.000000
max      380.000000
Name: 0, dtype: float64
```

Find transaction frequency of pack size

```
In [33]: pack_size.plot.hist()
plt.show()
```



Insight: 175g is found more followed by 150g pack

FREQUENCY OF PRODUCT BRAND NAMES

```
In [34]: df["Product"].str.split().str[0].value_counts()
```

```
Out[34]: Product
Kettle      41288
Smiths      27390
Pringles    25102
Doritos     22041
Thins       14075
RRD         11894
Infuzions   11057
WW          10320
Cobs        9693
Tostitos    9471
Twisties    9454
Tyrrells    6442
Grain       6272
Natural     6050
Cheezels    4603
CCs         4551
Red         4427
Dorito      3183
Infzns      3144
Smith       2963
Cheetos     2927
Snbts       1576
Burger      1564
Woolworths  1516
GrnWves     1468
Sunbites    1432
NCC         1419
French      1418
Name: count, dtype: int64
```

Add a new column for brand

```
In [35]: df["Brand"] = df["Product"].str.split().str[0]
df.head()
```

```
Out[35]:
```

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription	Brand
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium	Natural
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget	CCs
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget	Smiths
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S Cream Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget	Smiths
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget	Kettle

Combine same brands

```
In [36]: def clear_brand_names(line) :
brand = line["Brand"]
if brand == "Dorito" :
    return "Doritos"
elif brand == "GrnWves" or brand == "Grain" :
    return "Grain Waves"
elif brand == "Infzns" :
    return "Infuzions"
elif brand == "Natural" or brand == "NCC" :
    return "Natural Chip Co"
elif brand == "Red" :
    return "RRD"
elif brand == "Smith" :
    return "Smiths"
elif brand == "Snbts" :
    return "Sunbites"
elif brand == "WW" :
    return "Woolworths"
else :
    return brand
```

Cleaned transactional data

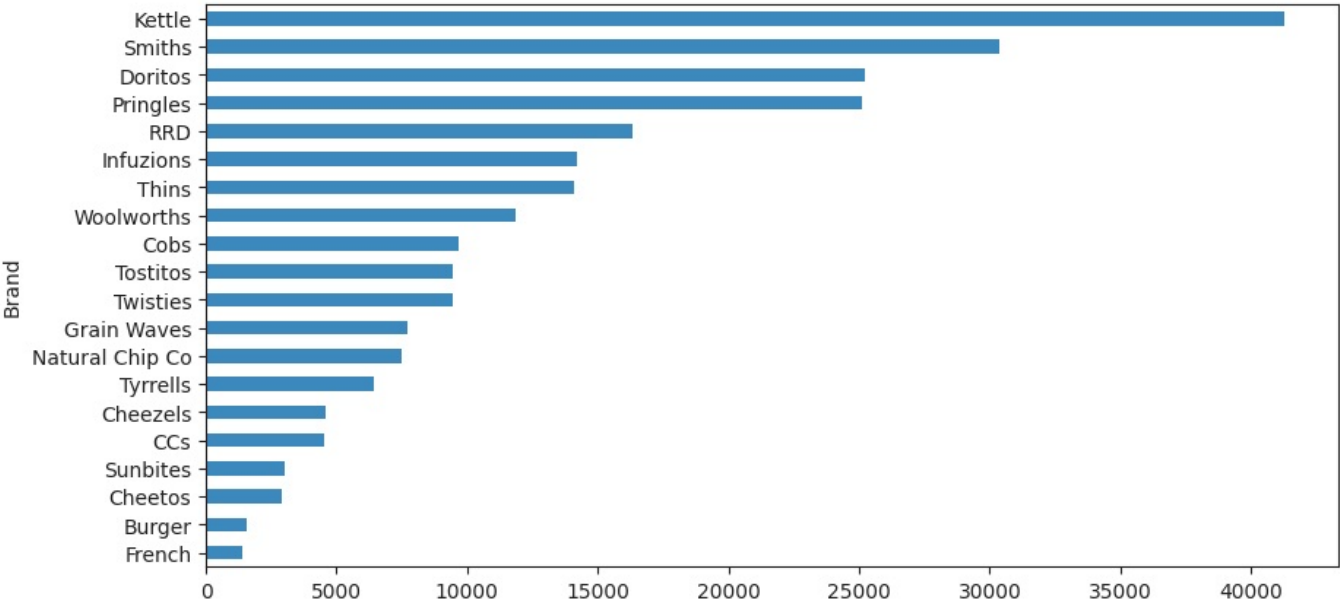
```
In [37]: df["Brand"] = df.apply(lambda line : clear_brand_names(line), axis=1)
df.head()
```

Out[37]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription	Brand
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium	Natural Chip Co
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget	CCs
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget	Smiths
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S Cream Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget	Smiths
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget	Kettle

Histogram on brand Frequency

```
In [38]: df["Brand"].value_counts(ascending=True).plot.barh(figsize = (10, 5))
plt.show()
```



Insight: "Kettle" is the most sold brand followed by Smiths

DATA ANALYSISWho spends the most on Chips(Total Sales), describing customers by lifestage and how is the subscription status

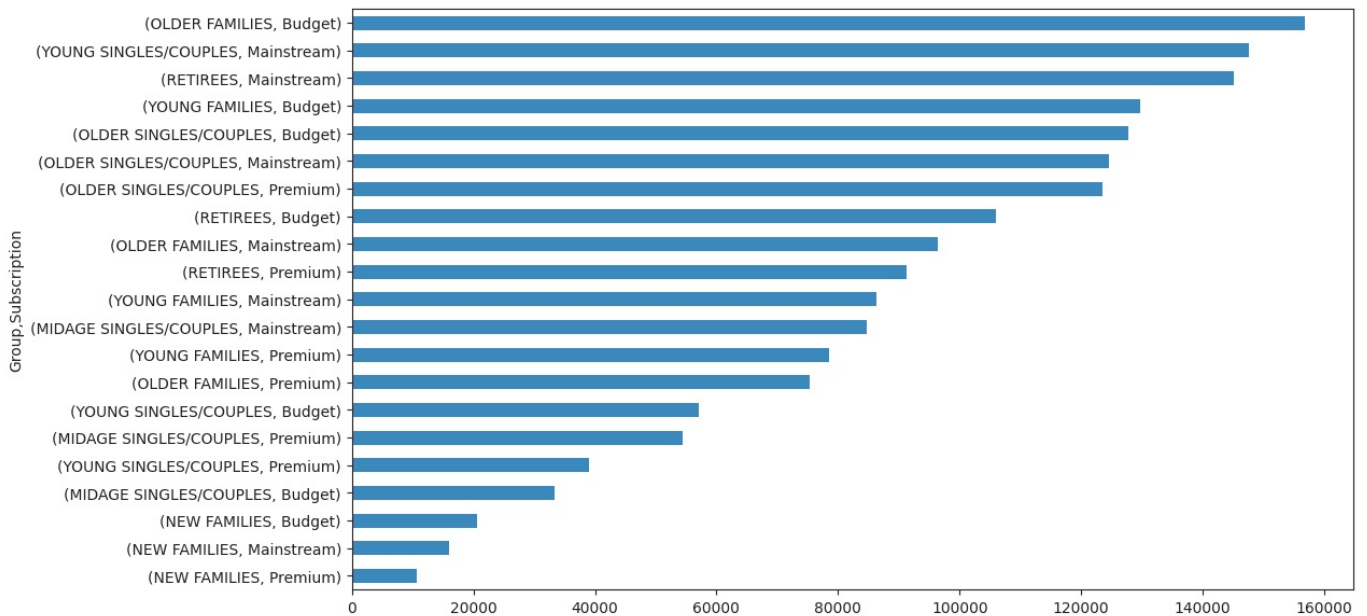
```
In [39]: most_shopping = merged_df.groupby(["Group", "Subscription"])["Sales"].agg(["sum"]).sort_values(ascending=False,
most_shopping
```

Out[39]:

Group	Subscription	sum
OLDER FAMILIES	Budget	156863.75
YOUNG SINGLES/COUPLES	Mainstream	147582.20
RETIREEES	Mainstream	145168.95
YOUNG FAMILIES	Budget	129717.95
OLDER SINGLES/COUPLES	Budget	127833.60
	Mainstream	124648.50
	Premium	123537.55
RETIREEES	Budget	105916.30
OLDER FAMILIES	Mainstream	96413.55
RETIREEES	Premium	91296.65
YOUNG FAMILIES	Mainstream	86338.25
MIDAGE SINGLES/COUPLES	Mainstream	84734.25
YOUNG FAMILIES	Premium	78571.70
OLDER FAMILIES	Premium	75242.60
YOUNG SINGLES/COUPLES	Budget	57122.10
MIDAGE SINGLES/COUPLES	Premium	54443.85
YOUNG SINGLES/COUPLES	Premium	39052.30
MIDAGE SINGLES/COUPLES	Budget	33345.70
	Mainstream	20607.45
	Premium	10760.80

Which customer segments contribute to chips sales

```
In [40]: most_shopping["sum"].sort_values().plot.barh(figsize = (12, 7))
plt.show()
```



Insight: Older Family with Budget Subscriptions spend more on chips

Who is the top contributor per group by subscription

```
In [41]: stage_agg_prem = df.groupby("Group")["Subscription"].agg(pd.Series.mode).sort_values()
print("Top contributor per Group by Subscription")
print(stage_agg_prem)
```

Top contributor per Group by Subscription

Group

NEW FAMILIES Budget

OLDER FAMILIES Budget

OLDER SINGLES/COUPLES Budget

YOUNG FAMILIES Budget

MIDAGE SINGLES/COUPLES Mainstream

RETIREEES Mainstream

YOUNG SINGLES/COUPLES Mainstream

Name: Subscription, dtype: object

How many customers are in each segment

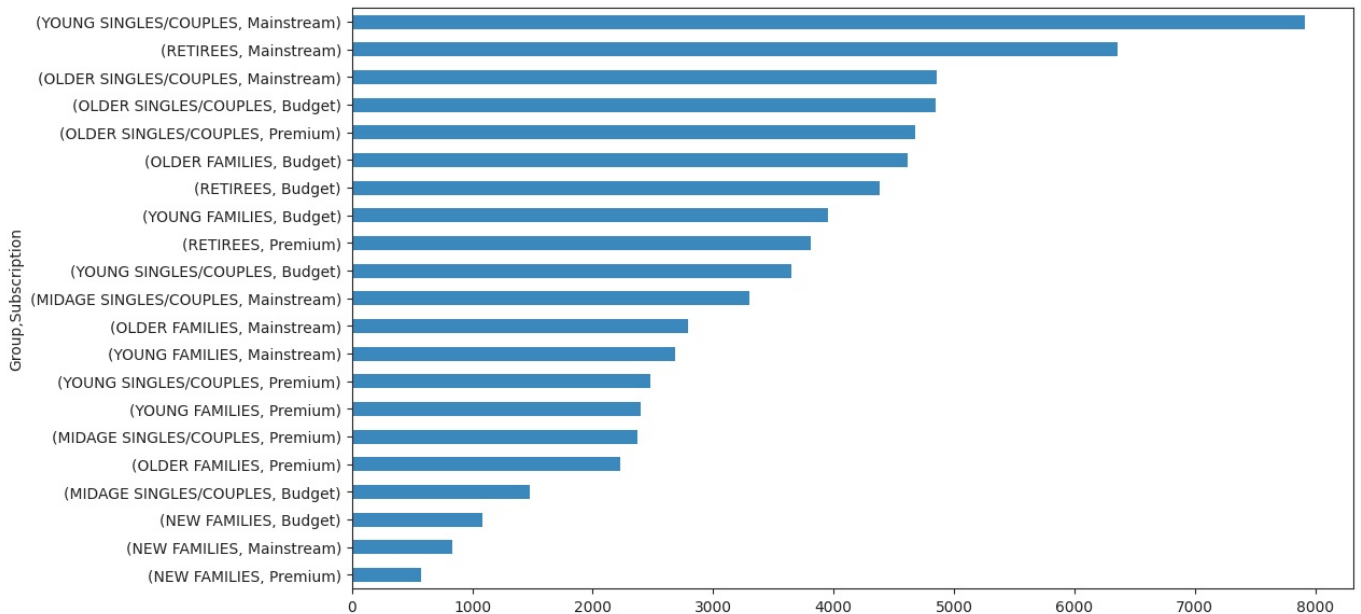
```
In [42]: cust_seg = df.groupby(["Group", "Subscription"])["Card No."].nunique().sort_values(ascending=False)
pd.DataFrame(cust_seg)
```

Out[42]:

		Card No.
Group	Subscription	
YOUNG SINGLES/COUPLES	Mainstream	7917
RETIREEES	Mainstream	6358
OLDER SINGLES/COUPLES	Mainstream	4858
	Budget	4849
	Premium	4682
OLDER FAMILIES	Budget	4611
RETIREEES	Budget	4385
YOUNG FAMILIES	Budget	3953
RETIREEES	Premium	3812
YOUNG SINGLES/COUPLES	Budget	3647
MIDAGE SINGLES/COUPLES	Mainstream	3298
OLDER FAMILIES	Mainstream	2788
YOUNG FAMILIES	Mainstream	2685
YOUNG SINGLES/COUPLES	Premium	2480
YOUNG FAMILIES	Premium	2398
MIDAGE SINGLES/COUPLES	Premium	2369
OLDER FAMILIES	Premium	2231
MIDAGE SINGLES/COUPLES	Budget	1474
NEW FAMILIES	Budget	1087
	Mainstream	830
	Premium	575

Customer Segment Frequency

```
In [43]: cust_seg.sort_values().plot.barh(figsize = (12, 7))
plt.show()
```



How many chips are bought per customer by segment

```
In [44]: chips_shop_per_segment = df.groupby(["Card No.", "Group", "Subscription"]).count()["Date"]
chips_shop_per_segment.to_frame()
```

Out[44]:

			Date
Card No.	Group	Subscription	
1000	YOUNG SINGLES/COUPLES	Premium	1
1002	YOUNG SINGLES/COUPLES	Mainstream	1
1003	YOUNG FAMILIES	Budget	2
1004	OLDER SINGLES/COUPLES	Mainstream	1
1005	MIDAGE SINGLES/COUPLES	Mainstream	1
...	...	...	...
2370651	MIDAGE SINGLES/COUPLES	Mainstream	1
2370701	YOUNG FAMILIES	Mainstream	1
2370751	YOUNG FAMILIES	Premium	1
2370961	OLDER FAMILIES	Budget	1
2373711	YOUNG SINGLES/COUPLES	Mainstream	1

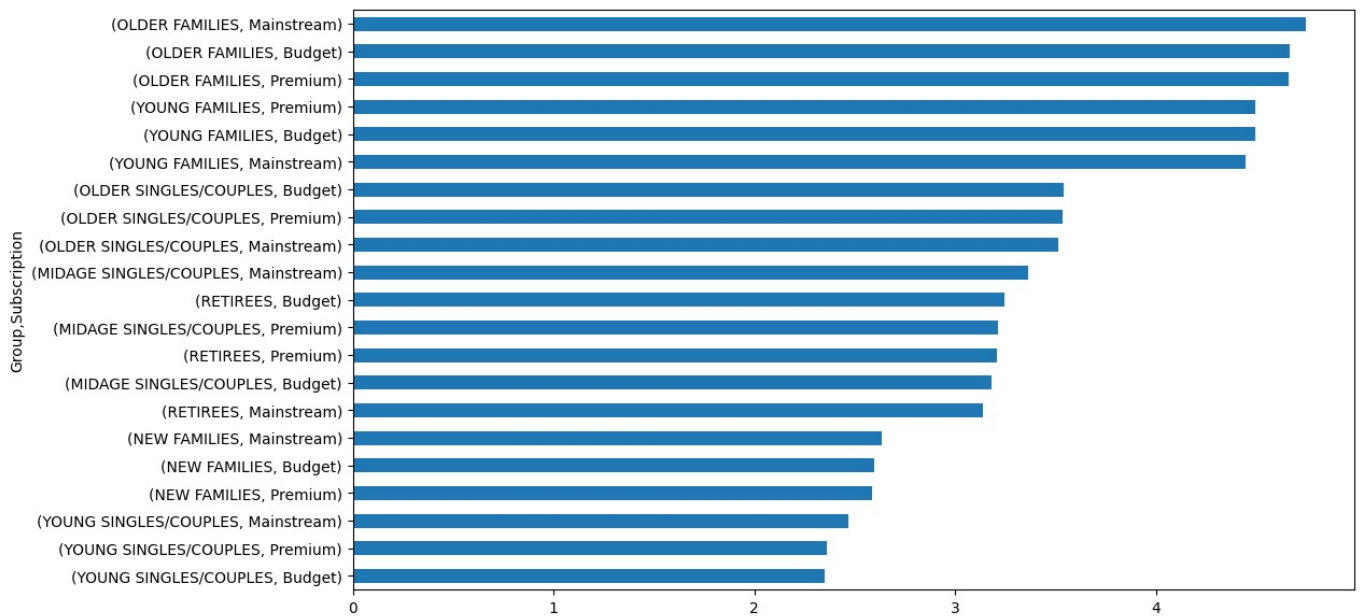
71287 rows × 4 columns

```
In [45]: segment_shop_chips = chips_shop_per_segment.groupby(["Group", "Subscription"]).agg(["mean"]).sort_values(ascending=True)
segment_shop_chips
```

Out[45]:

		mean
Group	Subscription	
OLDER FAMILIES	Mainstream	4.749283
	Budget	4.665799
	Premium	4.662931
YOUNG FAMILIES	Premium	4.497081
	Budget	4.493549
	Mainstream	4.449534
OLDER SINGLES/COUPLES	Budget	3.541349
	Premium	3.536950
	Mainstream	3.511939
MIDAGE SINGLES/COUPLES	Mainstream	3.364160
RETIREES	Budget	3.244014
MIDAGE SINGLES/COUPLES	Premium	3.213170
RETIREES	Premium	3.209864
MIDAGE SINGLES/COUPLES	Budget	3.182497
RETIREES	Mainstream	3.140925
NEW FAMILIES	Mainstream	2.632530
	Budget	2.597976
	Premium	2.587826
YOUNG SINGLES/COUPLES	Mainstream	2.468612
	Premium	2.359677
	Budget	2.350699

```
In [46]: segment_shop_chips["mean"].sort_values().plot.barh(figsize = (12, 7))
plt.show()
```

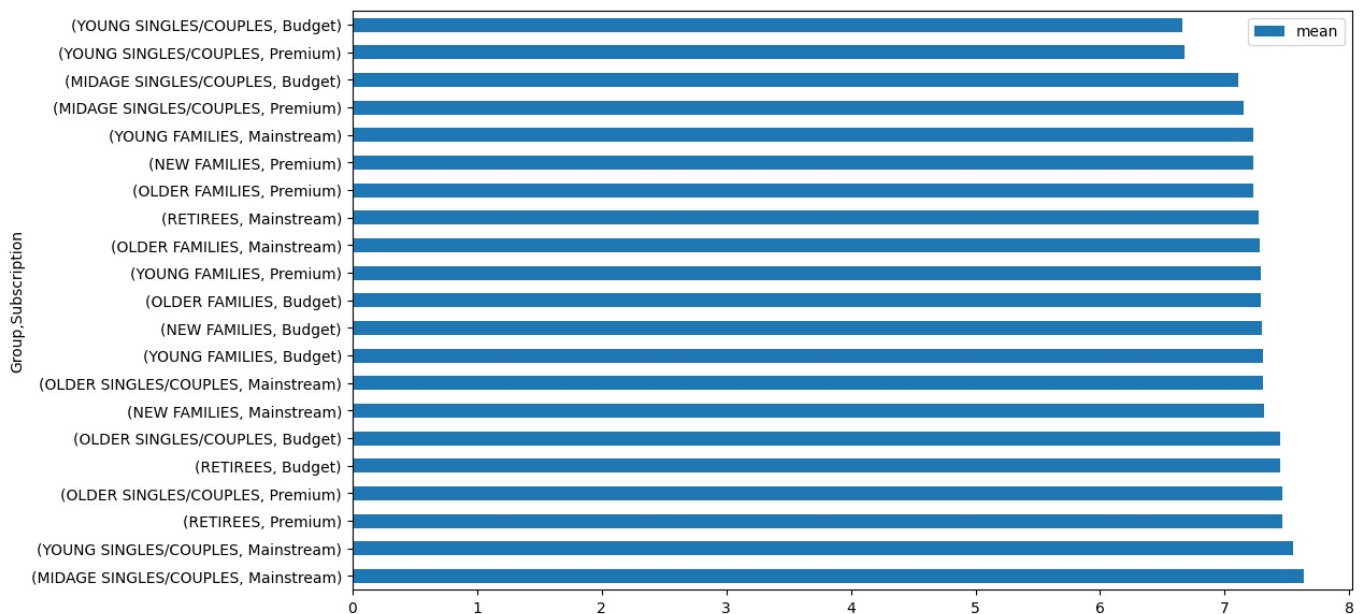
```
In [47]: avg_chips_price_cust_segment = df.groupby(["Group", "Subscription"])["Sales"].agg(["mean"]).sort_values(ascending=True)
avg_chips_price_cust_segment
```

Out[47]:

			mean
	Group	Subscription	
	MIDAGE SINGLES/COUPLES	Mainstream	7.637156
	YOUNG SINGLES/COUPLES	Mainstream	7.551279
	RETIREES	Premium	7.461315
	OLDER SINGLES/COUPLES	Premium	7.459997
	RETIREES	Budget	7.445786
	OLDER SINGLES/COUPLES	Budget	7.444305
	NEW FAMILIES	Mainstream	7.313364
	OLDER SINGLES/COUPLES	Mainstream	7.306049
	YOUNG FAMILIES	Budget	7.302705
	NEW FAMILIES	Budget	7.297256
	OLDER FAMILIES	Budget	7.291241
	YOUNG FAMILIES	Premium	7.285951
	OLDER FAMILIES	Mainstream	7.281440
	RETIREES	Mainstream	7.269352
	OLDER FAMILIES	Premium	7.232779
	NEW FAMILIES	Premium	7.231720
	YOUNG FAMILIES	Mainstream	7.226772
	MIDAGE SINGLES/COUPLES	Premium	7.152371
		Budget	7.108442
	YOUNG SINGLES/COUPLES	Premium	6.673325
		Budget	6.663023

```
In [48]: avg_chips_price_cust_segment.plot.barh(figsize = (12, 7))
```

```
Out[48]: <Axes: ylabel='Group,Subscription'>
```



```
In [49]: mainstream = df["Subscription"] == "Mainstream"
budget_premium = (df["Subscription"] == "Budget") | (df["Subscription"] == "Premium")
young_midage = (df["Group"] == "YOUNG SINGLES/COUPLES") | (df["Group"] == "MIDAGE SINGLES/COUPLES")

a = df[young_midage & mainstream]["Sales"]
b = df[young_midage & budget_premium]["Sales"]

stat, pval = ttest_ind(a.values, b.values, equal_var=False)
print(pval)
pval < 0.0000001
```

1.834645908180742e-237

Out[49]: np.True_

Group subscription and lifestage into segments

```
In [50]: df.groupby(["Group", "Subscription"])["Brand"].agg(pd.Series.mode).sort_values().to_frame()
```

Out[50]:

		Brand
Group	Subscription	
MIDAGE SINGLES/COUPLES	Budget	Kettle
YOUNG SINGLES/COUPLES	Budget	Kettle
YOUNG FAMILIES	Premium	Kettle
	Mainstream	Kettle
	Budget	Kettle
RETIREEES	Premium	Kettle
	Mainstream	Kettle
	Budget	Kettle
OLDER SINGLES/COUPLES	Premium	Kettle
YOUNG SINGLES/COUPLES	Mainstream	Kettle
OLDER SINGLES/COUPLES	Mainstream	Kettle
OLDER FAMILIES	Premium	Kettle
	Mainstream	Kettle
	Budget	Kettle
NEW FAMILIES	Premium	Kettle
	Mainstream	Kettle
	Budget	Kettle
MIDAGE SINGLES/COUPLES	Premium	Kettle
	Mainstream	Kettle
OLDER SINGLES/COUPLES	Budget	Kettle
YOUNG SINGLES/COUPLES	Premium	Kettle

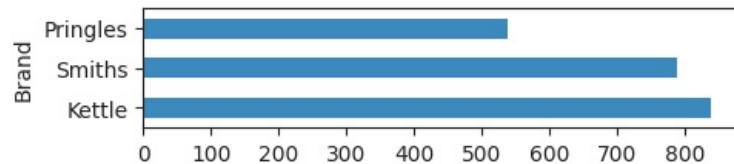
Kettle is the most purchased brand in each segment

Find which chips packet size is most frequently purchased by each segments.

```
In [51]: for stage in df["Group"].unique() :
          for subs in df["Subscription"].unique() :
              print("=====", stage, "-", subs, "=====")
              summary = df[(df["Group"] == stage) & (df["Subscription"] == subs)][["Brand"]].value_counts().head(3)
              print(summary)
              plt.figure()
              summary.plot.barh(figsize = (5, 1))
              plt.show()
```

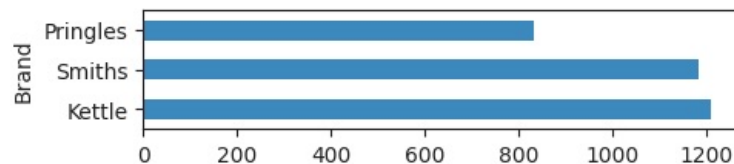
===== YOUNG SINGLES/COUPLES - Premium =====

```
Brand
Kettle      838
Smiths      787
Pringles    537
Name: count, dtype: int64
```



===== YOUNG SINGLES/COUPLES - Budget =====

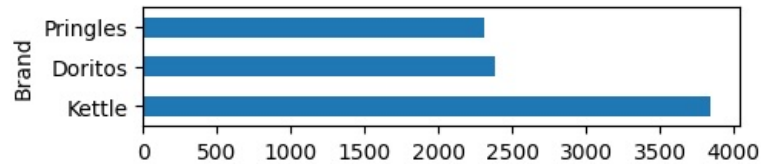
```
Brand
Kettle      1211
Smiths      1185
Pringles     832
Name: count, dtype: int64
```



===== YOUNG SINGLES/COUPLES - Mainstream =====

Brand
Kettle 3844
Doritos 2379
Pringles 2315

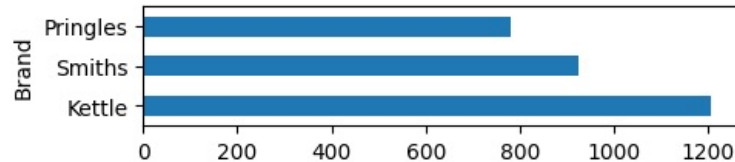
Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Premium =====

Brand
Kettle 1206
Smiths 923
Pringles 781

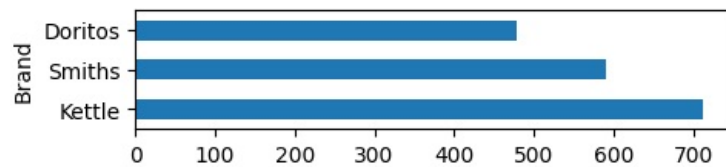
Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Budget =====

Brand
Kettle 713
Smiths 591
Doritos 479

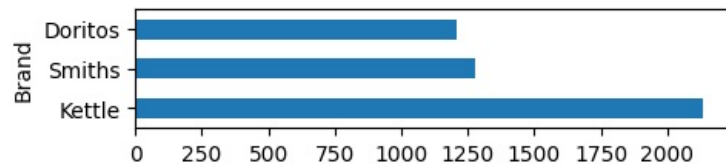
Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Mainstream =====

Brand
Kettle 2136
Smiths 1276
Doritos 1210

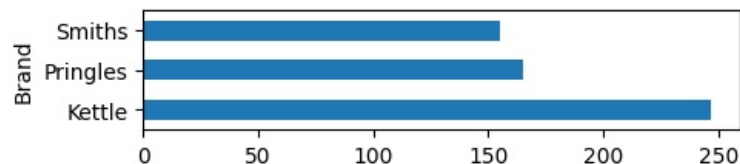
Name: count, dtype: int64



===== NEW FAMILIES - Premium =====

Brand
Kettle 247
Pringles 165
Smiths 155

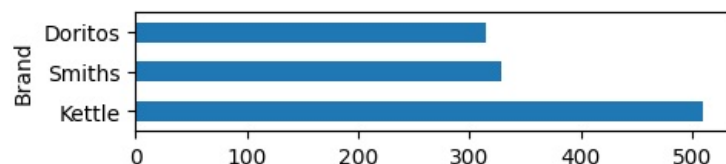
Name: count, dtype: int64



===== NEW FAMILIES - Budget =====

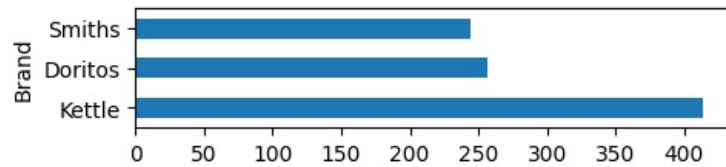
Brand
Kettle 510
Smiths 328
Doritos 315

Name: count, dtype: int64



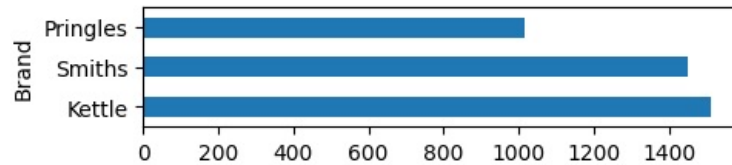
===== NEW FAMILIES - Mainstream =====

Brand
 Kettle 414
 Doritos 257
 Smiths 244
 Name: count, dtype: int64



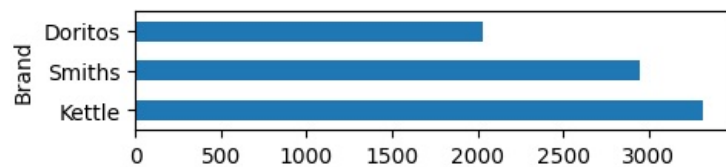
===== OLDER FAMILIES - Premium =====

Brand
 Kettle 1512
 Smiths 1448
 Pringles 1014
 Name: count, dtype: int64



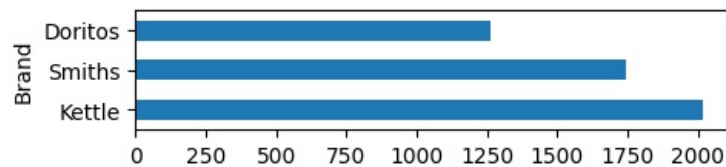
===== OLDER FAMILIES - Budget =====

Brand
 Kettle 3320
 Smiths 2948
 Doritos 2032
 Name: count, dtype: int64



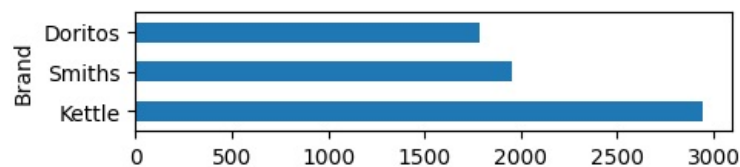
===== OLDER FAMILIES - Mainstream =====

Brand
 Kettle 2019
 Smiths 1742
 Doritos 1263
 Name: count, dtype: int64



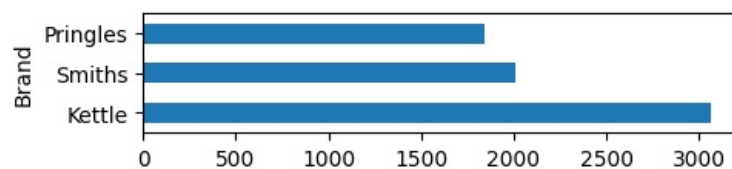
===== OLDER SINGLES/COUPLES - Premium =====

Brand
 Kettle 2947
 Smiths 1952
 Doritos 1784
 Name: count, dtype: int64



===== OLDER SINGLES/COUPLES - Budget =====

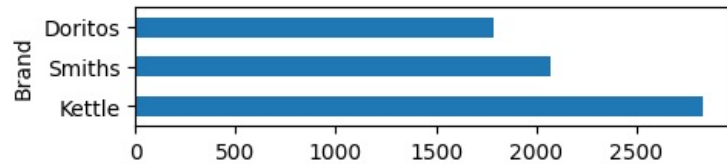
Brand
 Kettle 3065
 Smiths 2010
 Pringles 1843
 Name: count, dtype: int64



===== OLDER SINGLES/COUPLES - Mainstream =====

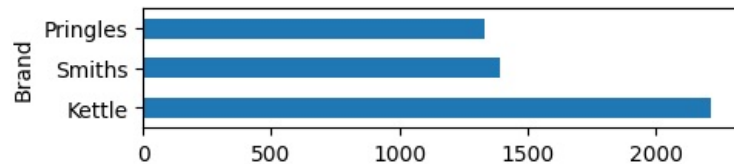
Brand
Kettle 2835

Smiths 2070
Doritos 1791
Name: count, dtype: int64



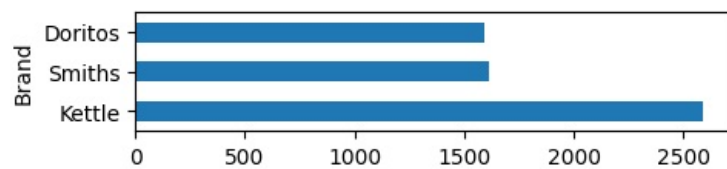
===== RETIREES - Premium =====

Brand
Kettle 2216
Smiths 1395
Pringles 1331
Name: count, dtype: int64



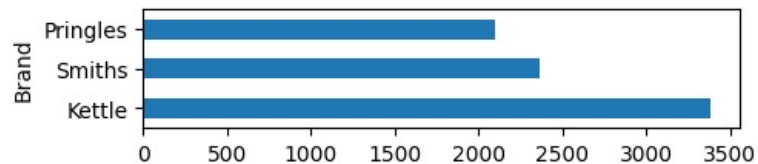
===== RETIREES - Budget =====

Brand
Kettle 2592
Smiths 1612
Doritos 1592
Name: count, dtype: int64



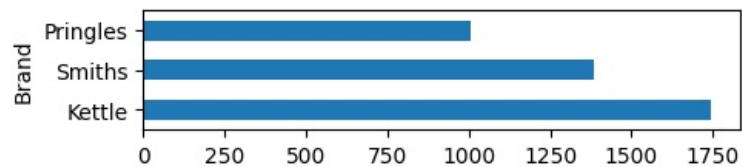
===== RETIREES - Mainstream =====

Brand
Kettle 3386
Smiths 2367
Pringles 2103
Name: count, dtype: int64



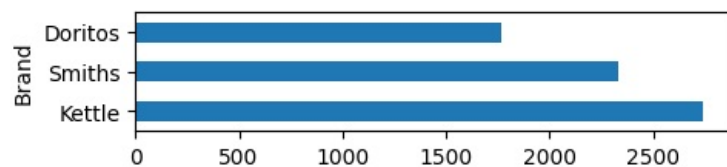
===== YOUNG FAMILIES - Premium =====

Brand
Kettle 1745
Smiths 1384
Pringles 1007
Name: count, dtype: int64



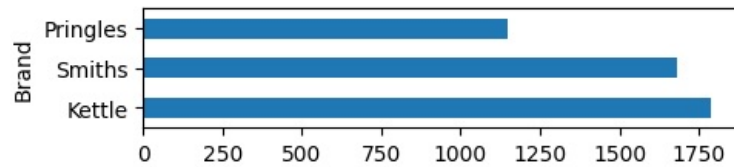
===== YOUNG FAMILIES - Budget =====

Brand
Kettle 2743
Smiths 2334
Doritos 1767
Name: count, dtype: int64



===== YOUNG FAMILIES - Mainstream =====

Brand
Kettle 1789
Smiths 1681
Pringles 1148
Name: count, dtype: int64



Insight:

"Kettle" as the most purchased brand in every segment. "YOUNG SINGLES/COUPLES Mainstream" Has "Smiths" as their second most purchased brand. "YOUNG SINGLES/COUPLES Mainstream" has "Doritos" as their second most purchased brand.

```
In [52]: merged_pack = pd.concat([df, pack_size.rename("Size")], axis=1) # MERGE DF WITH PACK SIZE
merged_pack.head()
```

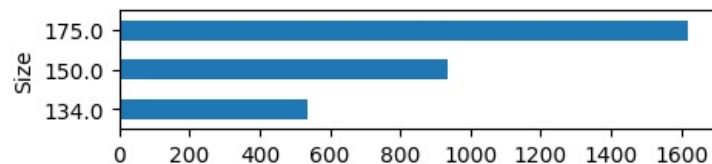
Out[52]:

	Date	Store Id	Card No.	Transaction Id	Product Id	Product	Quantity	Sales	Group	Subscription	Brand	Size
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	YOUNG SINGLES/COUPLES	Premium	Natural Chip Co	175.0
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	MIDAGE SINGLES/COUPLES	Budget	CCs	175.0
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	MIDAGE SINGLES/COUPLES	Budget	Smiths	170.0
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S Cream Onion 175g	5	15.0	MIDAGE SINGLES/COUPLES	Budget	Smiths	175.0
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny Jlpno Chili 150g	3	13.8	MIDAGE SINGLES/COUPLES	Budget	Kettle	150.0

```
In [53]: for stage in merged_pack["Group"].unique() :
          for subs in merged_pack["Subscription"].unique() :
              print("=====", stage, "-", subs, "=====")
              summary = merged_pack[(merged_pack["Group"] == stage) & (merged_pack["Subscription"] == subs)]["Size"].value_counts()
              print(summary)
              plt.figure()
              summary.plot.barh(figsize = (5, 1))
              plt.show()
```

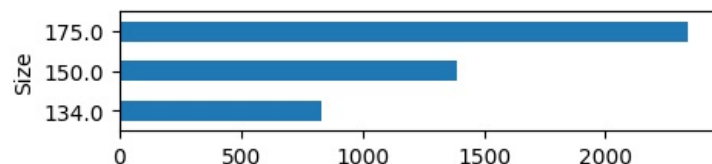
===== YOUNG SINGLES/COUPLES - Premium =====

Size
134.0 537
150.0 933
175.0 1618
Name: count, dtype: int64



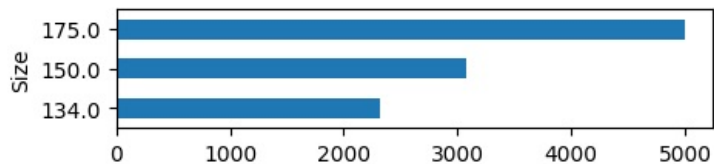
===== YOUNG SINGLES/COUPLES - Budget =====

Size
134.0 832
150.0 1390
175.0 2338
Name: count, dtype: int64



===== YOUNG SINGLES/COUPLES - Mainstream =====

Size
134.0 2315
150.0 3080
175.0 4997
Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Premium =====

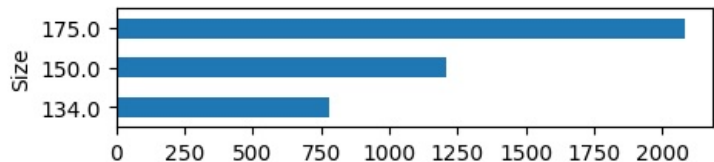
Size

134.0 781

150.0 1207

175.0 2082

Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Budget =====

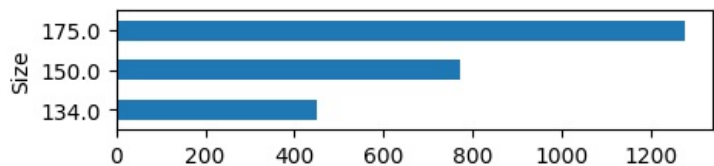
Size

134.0 449

150.0 771

175.0 1277

Name: count, dtype: int64



===== MIDAGE SINGLES/COUPLES - Mainstream =====

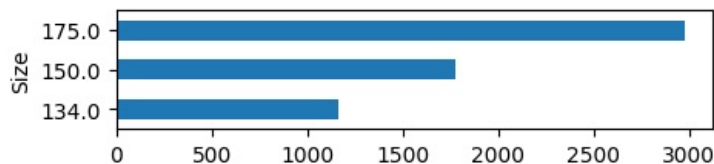
Size

134.0 1159

150.0 1777

175.0 2975

Name: count, dtype: int64



===== NEW FAMILIES - Premium =====

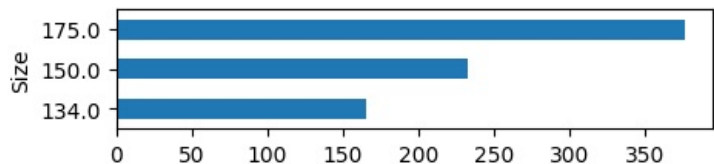
Size

134.0 165

150.0 233

175.0 376

Name: count, dtype: int64



===== NEW FAMILIES - Budget =====

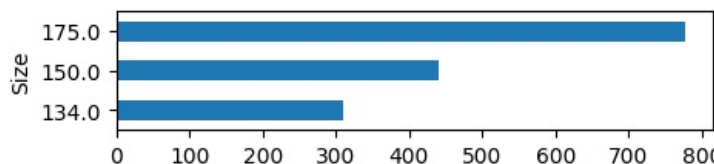
Size

134.0 309

150.0 440

175.0 777

Name: count, dtype: int64



===== NEW FAMILIES - Mainstream =====

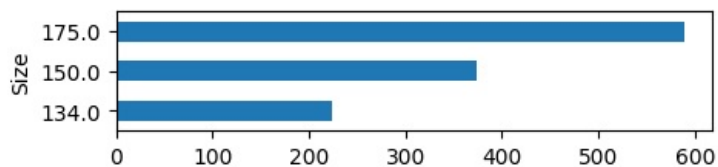
Size

134.0 224

150.0 374

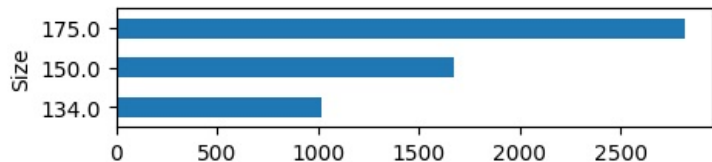
175.0 589

Name: count, dtype: int64



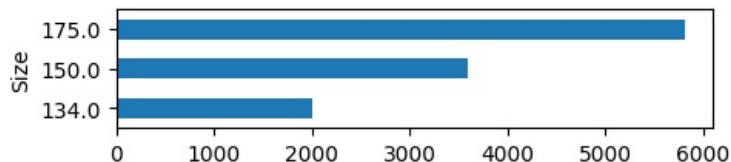
===== OLDER FAMILIES - Premium =====

```
Size
134.0    1014
150.0    1673
175.0    2816
Name: count, dtype: int64
```



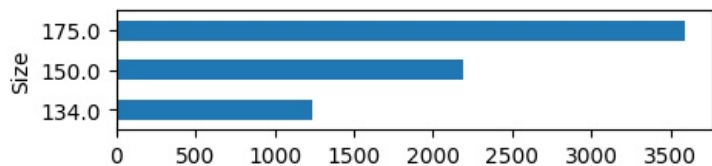
===== OLDER FAMILIES - Budget =====

```
Size
134.0    1996
150.0    3588
175.0    5808
Name: count, dtype: int64
```



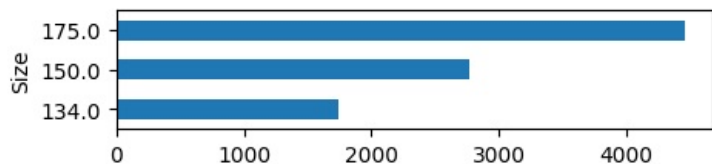
===== OLDER FAMILIES - Mainstream =====

```
Size
134.0    1234
150.0    2189
175.0    3588
Name: count, dtype: int64
```



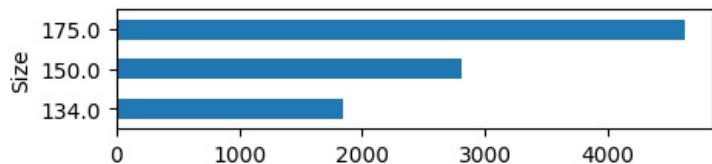
===== OLDER SINGLES/COUPLES - Premium =====

```
Size
134.0    1744
150.0    2768
175.0    4458
Name: count, dtype: int64
```



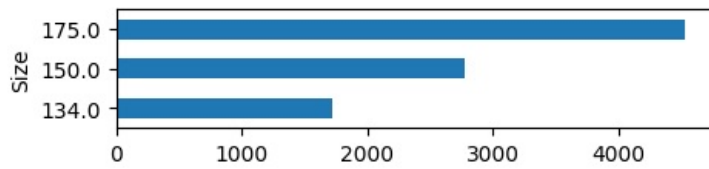
===== OLDER SINGLES/COUPLES - Budget =====

```
Size
134.0    1843
150.0    2811
175.0    4625
Name: count, dtype: int64
```



===== OLDER SINGLES/COUPLES - Mainstream =====

```
Size
134.0    1720
150.0    2773
175.0    4525
Name: count, dtype: int64
```



===== RETIREES - Premium =====

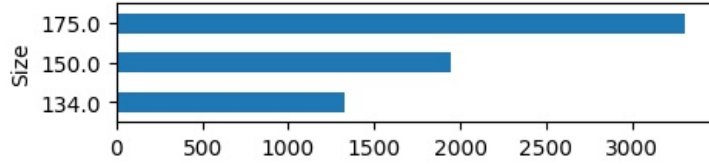
Size

134.0 1331

150.0 1943

175.0 3306

Name: count, dtype: int64



===== RETIREES - Budget =====

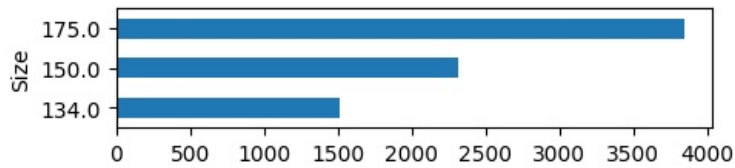
Size

134.0 1517

150.0 2319

175.0 3847

Name: count, dtype: int64



===== RETIREES - Mainstream =====

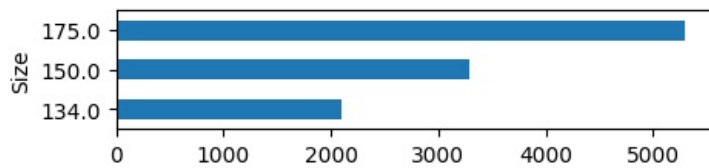
Size

134.0 2103

150.0 3290

175.0 5295

Name: count, dtype: int64



===== YOUNG FAMILIES - Premium =====

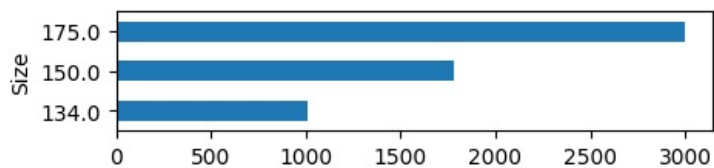
Size

134.0 1007

150.0 1778

175.0 2998

Name: count, dtype: int64



===== YOUNG FAMILIES - Budget =====

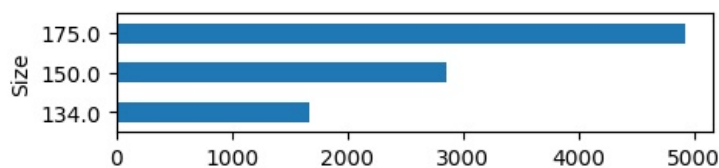
Size

134.0 1674

150.0 2862

175.0 4921

Name: count, dtype: int64



===== YOUNG FAMILIES - Mainstream =====

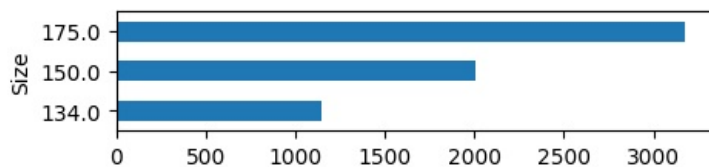
Size

134.0 1148

150.0 2004

175.0 3174

Name: count, dtype: int64



Insight : 175g is the most frequently purchased chips packet size followed by 150g packets for all segments.

Find average quantity of chips bought per segment

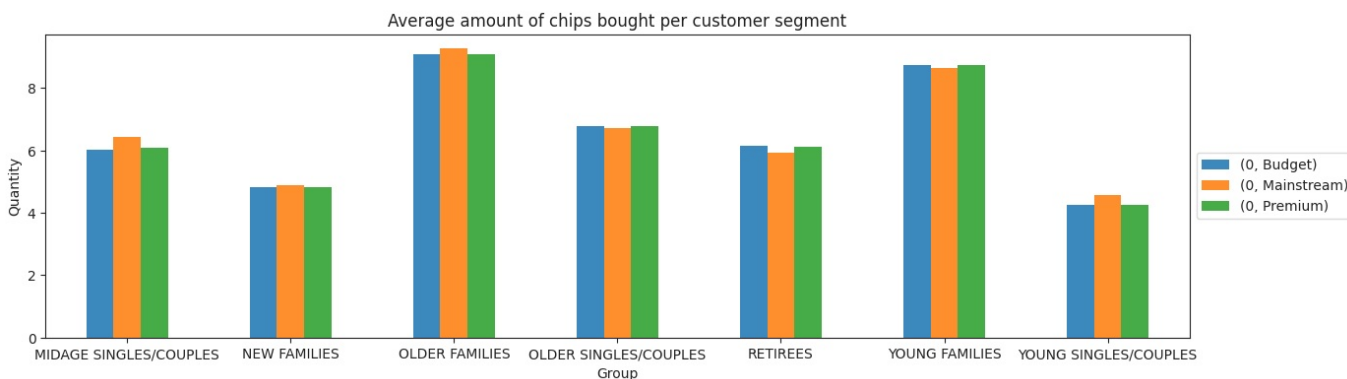
```
In [54]: avg_chips=(df.groupby(["Group","Subscription"])["Quantity"].sum()/df.groupby(["Group","Subscription"])["Card N
avg_chips
```

```
Out[54]: 0
```

Group	Subscription	
OLDER FAMILIES	Mainstream	9.255380
	Budget	9.076773
	Premium	9.071717
YOUNG FAMILIES	Budget	8.722995
	Premium	8.716013
	Mainstream	8.638361
OLDER SINGLES/COUPLES	Budget	6.781398
	Premium	6.769543
	Mainstream	6.712021
MIDAGE SINGLES/COUPLES	Mainstream	6.432080
	Budget	6.141847
	Premium	6.103358
RETIREES	Budget	6.103358
	Premium	6.078514
	Budget	6.026459
NEW FAMILIES	Mainstream	5.925920
	Budget	4.891566
	Premium	4.815652
YOUNG SINGLES/COUPLES	Mainstream	4.575597
	Premium	4.264113
	Budget	4.250069

Plot a histogram on average amount of chips bought per segment

```
In [55]: avg_chips.unstack().plot.bar(figsize = (15, 4), rot = 0)
plt.title("Average amount of chips bought per customer segment")
plt.legend(loc = "center left", bbox_to_anchor = (1.0, 0.5))
plt.ylabel("Quantity")
plt.show()
```



Insight: Older families with mainstream subscription purchases more quantity of chips

Find Average chips price per transaction by segments

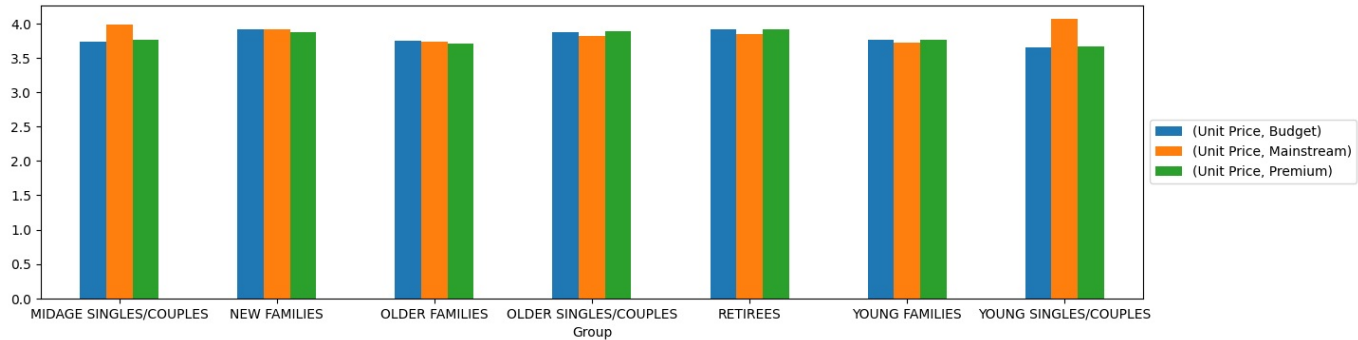
```
In [56]: df["Unit Price"] = df["Sales"] / df["Quantity"].replace(0, pd.NA) # Calculate "Unit Price" only where "Quantity" is not zero
chips_segment = df.groupby(["Group", "Subscription"], dropna=False)["Unit Price"].mean().sort_values(ascending=True)
chips_segment # Display the resulting DataFrame
```

Out[56]:

Unit Price		
Group	Subscription	
YOUNG SINGLES/COUPLES	Mainstream	4.065642
MIDAGE SINGLES/COUPLES	Mainstream	3.994241
RETIREEES	Budget	3.924404
	Premium	3.920942
NEW FAMILIES	Budget	3.917688
	Mainstream	3.916133
OLDER SINGLES/COUPLES	Premium	3.893182
	Budget	3.882096
NEW FAMILIES	Premium	3.872110
RETIREEES	Mainstream	3.844294
OLDER SINGLES/COUPLES	Mainstream	3.814665
MIDAGE SINGLES/COUPLES	Premium	3.770698
YOUNG FAMILIES	Premium	3.762150
	Budget	3.760737
OLDER FAMILIES	Budget	3.745340
MIDAGE SINGLES/COUPLES	Budget	3.743328
OLDER FAMILIES	Mainstream	3.737077
YOUNG FAMILIES	Mainstream	3.724533
OLDER FAMILIES	Premium	3.717000
	Budget	3.657366
YOUNG SINGLES/COUPLES	Premium	3.665414

Insight: Mainstream midage and young singles and couples are more willing to pay more per packet of chips compared to their budget and premium counterparts.

```
In [57]: chips_segment.unstack().plot.bar(figsize = (15, 4), rot = 0)
plt.legend(loc = "center left", bbox_to_anchor = (1.0, 0.5))
plt.show()
```



Insights and Recommendations 1. Older Families - Budget Segment - Strength: Frequent Purchases - Introduce promotions that incentivize repeat buying to increase purchase frequency. - Strength: High Quantity per Visit - Offer bundle deals or discounts on larger purchases to encourage bulk buying. 2. Young Singles/Couples - Mainstream Segment - Brand Preference Insight - This group uniquely ranks Doritos as their second most purchased brand (after Kettle). - Collaborate with Doritos for targeted branding campaigns tailored to this segment. - Strength: Large Population Base - Prioritize high visibility and frequent promotional outreach to maximize engagement. 3. Retirees - Mainstream Segment - Strength: Large Population Base - Since retirees contribute significantly to overall sales volume, ensure promotions are widely distributed and repeated often to capture their attention. 4. General Insights - All Segments - Top Brand: Kettle is consistently the most purchased brand across all groups. - Preferred Pack Sizes: 175g packs are most popular, followed by 150g. - Promotional Strategy: Highlight these preferences in broad campaigns to appeal universally across segments.

```
In [ ]:
```